

riko: A stream processing engine modeled after Yahoo! Pipes

Index

[Introduction](#) | [Requirements](#) | [Word Count](#) | [Motivation](#) | [Usage](#) | [Installation](#) | [Design Principles](#) | [Scripts](#) | [Command-line Interface](#) | [Contributing](#) | [Credits](#) | [More Info](#) | [Project Structure](#) | [License](#)

Introduction

riko is a pure Python [library](#) for analyzing and processing streams of structured data. riko has [synchronous](#) and [asynchronous](#) APIs, supports [parallel execution](#), and is well suited for processing RSS feeds ¹. riko also supplies a [command-line interface](#) for executing flows, i.e., stream processors aka workflows.

With riko, you can

- Read csv/xml/json/html files
- Create text and data based flows via modular [pipes](#)
- Parse, extract, and process RSS/Atom feeds
- Create awesome mashups ², APIs, and maps
- Perform [parallel processing](#) via cpus/processors or threads
- and much more...

Notes

Requirements

riko has been tested and is known to work on Python 3.7, 3.8, and 3.9; and PyPy3.7.

Optional Dependencies

Feature	Dependency	Installation
Async API	Twisted	<code>pip install riko[async]</code>
Accelerated xml parsing	lxml ³	<code>pip install riko[xml]</code>
Accelerated feed parsing	speedparser ⁴	<code>pip install riko[xml]</code>

Notes

Word Count

In this example, we use several [pipes](#) to count the words on a webpage.

```
>>> ### Create a SyncPipe flow ###
>>> #
>>> # `SyncPipe` is a convenience class that creates chainable flows
>>> # and allows for parallel processing.
```

```

>>> from riko.collections import SyncPipe
>>>
>>> ### Set the pipe configurations ###
>>> #
>>> # Notes:
>>> # 1. the `detag` option will strip all html tags from the result
>>> # 2. fetch the text contained inside the 'body' tag of the hackernews
>>> #    homepage
>>> # 3. replace newlines with spaces and assign the result to 'content'
>>> # 4. tokenize the resulting text using whitespace as the delimiter
>>> # 5. count the number of times each token appears
>>> # 6. obtain the raw stream
>>> # 7. extract the first word and its count
>>> # 8. extract the second word and its count
>>> # 9. extract the third word and its count
>>> url = 'https://news.ycombinator.com/'
>>> fetch_conf = {
...     'url': url, 'start': '<body>', 'end': '</body>', 'detag': True} # 1
>>>
>>> replace_conf = {
...     'rule': [
...         {'find': '\r\n', 'replace': ' '},
...         {'find': '\n', 'replace': ' '}]
... }
>>>
>>> flow = (
...     SyncPipe('fetchpage', conf=fetch_conf) # 2
...     .strreplace(conf=replace_conf, assign='content') # 3
...     .tokenizer(conf={'delimiter': ' '}, emit=True) # 4
...     .count(conf={'count_key': 'content'}) # 5
... )
>>>
>>> stream = flow.output # 6
>>> next(stream) # 7
{'sad': 1}
>>> next(stream) # 8
{'(': 28}
>>> next(stream) # 9
{'(1999)': 1}

```

Motivation

Why I built riko

Yahoo! Pipes⁵ was a user friendly web application used to

aggregate, manipulate, and mashup content from around the web

Wanting to create custom pipes, I came across [pipe2py](#) which translated a Yahoo! Pipe into python code. pipe2py suited my needs at the time but was unmaintained and lacked asynchronous or parallel processing.

riko addresses the shortcomings of pipe2py but removed support for importing Yahoo! Pipes json workflows. riko contains ~ 40 built-in modules, aka pipes, that allow you to programatically perform most of the tasks Yahoo! Pipes allowed.

Why you should use riko

riko provides a number of benefits / differences from other stream processing applications such as Huginn, Flink, Spark, and Storm⁶. Namely:

- a small footprint (CPU and memory usage)
- native RSS/Atom support
- simple installation and usage
- a pure python library with [pypy](#) support
- builtin modular pipes to filter, sort, and modify streams

The subsequent tradeoffs riko makes are:

- not distributed (able to run on a cluster of servers)
- no GUI for creating flows
- doesn't continually monitor streams for new data
- can't react to specific events
- iterator (pull) based so streams only support a single consumer ⁷

The following table summarizes these observations:

library	Stream Type	Footprint	RS S	simple ⁸	asy nc	parallel	CEP ⁹	distributed
riko	pull	small	√	√	√	√		
pipe2py	pull	small	√	√				
Huginn	push	med	√		¹⁰	√	√	
Others	push	large	¹¹	¹²	¹³	√	√	√

For more detailed information, please check-out the [FAQ](#).

Notes

Usage

riko is intended to be used directly as a Python library.

Usage Index

- [Fetching feeds](#)
- [Synchronous processing](#)
- [Parallel processing](#)
- [Asynchronous processing](#)
- [Cookbook](#)

Fetching feeds

riko can fetch rss feeds from both local and remote filepaths via "source" pipes. Each "source" pipe returns a stream, i.e., an iterator of dictionaries, aka items.

```
>>> from riko.modules import fetch, fetchsitefeed
>>>
>>> ### Fetch an RSS feed ###
>>> stream = fetch.pipe(conf={'url': 'https://news.ycombinator.com/rss'})
>>>
```

```

>>> ### Fetch the first RSS feed found ###
>>> stream = fetchsitefeed.pipe(conf={'url': 'http://arstechnica.com/rss-feeds/'})
>>>
>>> ### View the fetched RSS feed(s) ###
>>> #
>>> # Note: regardless of how you fetch an RSS feed, it will have the same
>>> # structure
>>> item = next(stream)
>>> item.keys()
dict_keys(['title_detail', 'author.uri', 'tags', 'summary_detail', 'author_detail',
          'author.name', 'y:published', 'y:title', 'content', 'title', 'pubDate',
          'guidislink', 'id', 'summary', 'dc:creator', 'authors', 'published_parsed',
          'links', 'y:id', 'author', 'link', 'published'])

>>> item['title'], item['author'], item['id']
('Gravity doesn't care about quantum spin',
 'Chris Lee',
 'http://arstechnica.com/?p=924009')

```

Please see the [FAQ](#) for a complete list of supported [file types](#) and [protocols](#). Please see [Fetching data and feeds](#) for more examples.

Synchronous processing

riko can modify streams via the [40 built-in pipes](#)

```

>>> from riko.collections import SyncPipe
>>>
>>> ### Set the pipe configurations ###
>>> fetch_conf = {'url': 'https://news.ycombinator.com/rss'}
>>> filter_rule = {'field': 'link', 'op': 'contains', 'value': '.com'}
>>> xpath = '/html/body/center/table/tr[3]/td/table[2]/tr[1]/td/table/tr/td[3]/span/span'
>>> xpath_conf = {'url': {'subkey': 'comments'}, 'xpath': xpath}
>>>
>>> ### Create a SyncPipe flow ###
>>> #
>>> # `SyncPipe` is a convenience class that creates chainable flows
>>> # and allows for parallel processing.
>>> #
>>> # The following flow will:
>>> # 1. fetch the hackernews RSS feed
>>> # 2. filter for items with '.com' in the link
>>> # 3. sort the items ascending by title
>>> # 4. fetch the first comment from each item
>>> # 5. flatten the result into one raw stream
>>> # 6. extract the first item's content
>>> #
>>> # Note: sorting is not lazy so take caution when using this pipe
>>>
>>> flow = (
...     SyncPipe('fetch', conf=fetch_conf)           # 1
...     .filter(conf={'rule': filter_rule})         # 2
...     .sort(conf={'rule': {'sort_key': 'title'}}) # 3
...     .xpathfetchpage(conf=xpath_conf)            # 4
>>>
>>> stream = flow.output                             # 5
>>> next(stream)['content']                          # 6
'Open Artificial Pancreas home:'

```

Please see [alternate workflow creation](#) for an alternative (function based) method for creating a stream. Please see [pipes](#) for a complete list of available pipes.

Parallel processing

An example using riko's parallel API to spawn a ThreadPool ¹⁴

```
>>> from riko.collections import SyncPipe
>>>
>>> ### Set the pipe configurations ###
>>> fetch_conf = {'url': 'https://news.ycombinator.com/rss'}
>>> filter_rule = {'field': 'link', 'op': 'contains', 'value': '.com'}
>>> xpath = '/html/body/center/table/tr[3]/td/table[2]/tr[1]/td/table/tr/td[3]/span/span'
>>> xpath_conf = {'url': {'subkey': 'comments'}, 'xpath': xpath}
>>>
>>> ### Create a parallel SyncPipe flow ###
>>> #
>>> # The following flow will:
>>> # 1. fetch the hackernews RSS feed
>>> # 2. filter for items with '.com' in the article link
>>> # 3. fetch the first comment from all items in parallel (using 4 workers)
>>> # 4. flatten the result into one raw stream
>>> # 5. extract the first item's content
>>> #
>>> # Note: no point in sorting after the filter since parallel fetching doesn't guarantee
>>> # order
>>> flow = (
...     SyncPipe('fetch', conf=fetch_conf, parallel=True, workers=4) # 1
...     .filter(conf={'rule': filter_rule}) # 2
...     .xpathfetchpage(conf=xpath_conf) # 3
>>>
>>> stream = flow.output # 4
>>> next(stream)['content'] # 5
'He uses the following example for when to throw your own errors:'
```

Asynchronous processing

To enable asynchronous processing, you must install the async module.

```
pip install riko[async]
```

An example using riko's asynchronous API.

```
>>> from riko.bado import coroutine, react
>>> from riko.collections import AsyncPipe
>>>
>>> ### Set the pipe configurations ###
>>> fetch_conf = {'url': 'https://news.ycombinator.com/rss'}
>>> filter_rule = {'field': 'link', 'op': 'contains', 'value': '.com'}
>>> xpath = '/html/body/center/table/tr[3]/td/table[2]/tr[1]/td/table/tr/td[3]/span/span'
>>> xpath_conf = {'url': {'subkey': 'comments'}, 'xpath': xpath}
>>>
>>> ### Create an AsyncPipe flow ###
>>> #
>>> # The following flow will:
>>> # 1. fetch the hackernews RSS feed
>>> # 2. filter for items with '.com' in the article link
>>> # 3. asynchronously fetch the first comment from each item (using 4 connections)
>>> # 4. flatten the result into one raw stream
>>> # 5. extract the first item's content
>>> #
>>> # Note: no point in sorting after the filter since async fetching doesn't guarantee
>>> # order
```

```

>>> @coroutine
... def run(reactor):
...     stream = yield (
...         AsyncPipe('fetch', conf=fetch_conf, connections=4) # 1
...         .filter(conf={'rule': filter_rule}) # 2
...         .xpathfetchpage(conf=xpath_conf) # 3
...         .output) # 4
...     print(next(stream)['content']) # 5
>>>
>>> try:
...     react(run)
... except SystemExit:
...     pass
Here's how iteration works ():

```

Cookbook

Please see the [cookbook](#) or [ipython notebook](#) for more examples.

Notes

Installation

(You are using a [virtualenv](#), right?)

At the command line, install riko using either `pip` (*recommended*)

```
pip install riko
```

or `easy_install`

```
easy_install riko
```

Please see the [installation doc](#) for more details.

Design Principles

The primary data structures in riko are the item and stream. An item is just a python dictionary, and a stream is an iterator of items. You can create a stream manually with something as simple as `[{'content': 'hello world'}]`. You manipulate streams in riko via pipes. A pipe is simply a function that accepts either a stream or item, and returns a stream. pipes are composable: you can use the output of one pipe as the input to another pipe.

riko pipes come in two flavors; operators and processors. operators operate on an entire stream at once and are unable to handle individual items. Example operators include count, filter, and reverse.

```

>>> from riko.modules.reverse import pipe
>>>
>>> stream = [{'title': 'riko pt. 1'}, {'title': 'riko pt. 2'}]
>>> next(pipe(stream))
{'title': 'riko pt. 2'}

```

processors process individual items and can be parallelized across threads or processes. Example processors include fetchsitefeed, hash, itembuilder, and regex.

```
>>> from riko.modules.hash import pipe
>>>
>>> item = {'title': 'riko pt. 1'}
>>> stream = pipe(item, field='title')
>>> next(stream)
{'title': 'riko pt. 1', 'hash': 2853617420}
```

Some processors, e.g., tokenizer, return multiple results.

```
>>> from riko.modules.tokenizer import pipe
>>>
>>> item = {'title': 'riko pt. 1'}
>>> tokenizer_conf = {'delimiter': ' '}
>>> stream = pipe(item, conf=tokenizer_conf, field='title')
>>> next(stream)
{'tokenizer': [{'content': 'riko'},
               {'content': 'pt.'},
               {'content': '1'}],
 'title': 'riko pt. 1'}

>>> # In this case, if we just want the result, we can `emit` it instead
>>> stream = pipe(item, conf=tokenizer_conf, field='title', emit=True)
>>> next(stream)
{'content': 'riko'}
```

operators are split into sub-types of aggregators and composers. aggregators, e.g., count, combine all items of an input stream into a new stream with a single item; while composers, e.g., filter, create a new stream containing some or all items of an input stream.

```
>>> from riko.modules.count import pipe
>>>
>>> stream = [{'title': 'riko pt. 1'}, {'title': 'riko pt. 2'}]
>>> next(pipe(stream))
{'count': 2}
```

In case you are confused from the "Word Count" example up top, count can return multiple items if you pass in the count_key config option.

```
>>> counted = pipe(stream, conf={'count_key': 'title'})
>>> next(counted)
{'riko pt. 1': 1}
>>> next(counted)
{'riko pt. 2': 1}
```

processors are split into sub-types of source and transformer. sources, e.g., itembuilder, can create a stream while transformers, e.g. hash can only transform items in a stream.

```
>>> from riko.modules.itembuilder import pipe
>>>
>>> attrs = {'key': 'title', 'value': 'riko pt. 1'}
>>> next(pipe(conf={'attrs': attrs}))
{'title': 'riko pt. 1'}
```

The following table summaries these observations:

type	sub-type	input	output	parallelizable?	creates streams?
------	----------	-------	--------	-----------------	------------------

operator	aggregator	stream	stream ¹⁵		
	composer	stream	stream		
processor	source	item	stream	√	√
	transformer	item	stream	√	

If you are unsure of the type of pipe you have, check its metadata.

```
>>> from riko.modules import fetchpage, count
>>>
>>> fetchpage.async_pipe.__dict__
{'type': 'processor', 'name': 'fetchpage', 'sub_type': 'source'}
>>> count.pipe.__dict__
{'type': 'operator', 'name': 'count', 'sub_type': 'aggregator'}
```

The SyncPipe and AsyncPipe classes (among other things) perform this check for you to allow for convenient method chaining and transparent parallelization.

```
>>> from riko.collections import SyncPipe
>>>
>>> attrs = [
...     {'key': 'title', 'value': 'riko pt. 1'},
...     {'key': 'content', 'value': "Let's talk about riko!"}]
>>> flow = SyncPipe('itembuilder', conf={'attrs': attrs}).hash()
>>> flow.list[0]
{'title': 'riko pt. 1',
 'content': "Let's talk about riko!",
 'hash': 1346301218}
```

Please see the [cookbook](#) for advanced examples including how to wire in vales from other pipes or accept user input.

Notes

Command-line Interface

riko provides a command, runpipe, to execute workflows. A workflow is simply a file containing a function named pipe that creates a flow and processes the resulting stream.

CLI Usage

usage: runpipe [pipeid]

description: Runs a riko pipe

positional arguments:

pipeid The pipe to run (default: reads from stdin).

optional arguments:

-h, --help

show this help message and exit

-a, --async

Load async pipe.

-t, --test

Run in test mode (uses default inputs).

CLI Setup

flow.py


```

from riko.collections import SyncPipe

conf1 = {'attrs': [{'value': 'https://google.com', 'key': 'content'}]}
conf2 = {'rule': [{'find': 'com', 'replace': 'co.uk'}]}

def pipe(test=False):
    kwargs = {'conf': conf1, 'test': test}
    flow = SyncPipe('itembuilder', **kwargs).strreplace(conf=conf2)
    stream = flow.output

    for i in stream:
        print(i)

```

CLI Examples

Now to execute `flow.py`, type the command `runpipe flow`. You should then see the following output in your terminal:

```
https://google.co.uk
```

`runpipe` will also search the examples directory for workflows. Type `runpipe demo` and you should see the following output:

```
Deadline to clear up health law eligibility near 682
```

Scripts

`riko` comes with a built in task manager `manage`.

Setup

```
pip install riko[develop]
```

Examples

Run python linter and nose tests

```
manage lint
manage test
```

Contributing

Please mimic the coding style/conventions used in this repo. If you add new classes or functions, please add the appropriate doc blocks with examples. Also, make sure the python linter and nose tests pass.

Please see the [contributing doc](#) for more details.

Credits

Shoutout to [pipe2py](#) for heavily inspiring `riko`. `riko` started out as a fork of `pipe2py`, but has since diverged so much that little (if any) of the original code-base remains.

More Info

- [FAQ](#)
- [Cookbook](#)
- [iPython Notebook](#)
- [Step-by-Step Intro. Tutorial](#)

Project Structure

```

■■■■ benchmarks
■   ■■■■ __init__.py
■   ■■■■ parallel.py
■■■■ bin
■   ■■■■ run
■■■■ data/*
■■■■ docs
■   ■■■■ AUTHORS.rst
■   ■■■■ CHANGES.rst
■   ■■■■ COOKBOOK.rst
■   ■■■■ FAQ.rst
■   ■■■■ INSTALLATION.rst
■   ■■■■ TODO.rst
■■■■ examples/*
■■■■ helpers/*
■■■■ riko
■   ■■■■ __init__.py
■   ■■■■ lib
■       ■   ■■■■ __init__.py
■       ■   ■■■■ autorss.py
■       ■   ■■■■ collections.py
■       ■   ■■■■ dotdict.py
■       ■   ■■■■ log.py
■       ■   ■■■■ tags.py
■       ■   ■■■■ py
■   ■■■■ modules/*
■   ■■■■ twisted
■       ■   ■■■■ __init__.py
■       ■   ■■■■ collections.py
■       ■   ■■■■ py
■■■■ tests
■   ■■■■ __init__.py
■   ■■■■ standard.rc
■   ■■■■ test_examples.py
■■■■ CONTRIBUTING.rst
■■■■ dev-requirements.txt
■■■■ LICENSE
■■■■ Makefile
■■■■ manage.py
■■■■ MANIFEST.in
■■■■ optional-requirements.txt
■■■■ py2-requirements.txt
■■■■ README.rst
■■■■ requirements.txt
■■■■ setup.cfg
■■■■ setup.py
■■■■ tox.ini
```

License

riko is distributed under the [MIT License](#).

1 Really Simple Syndication
2 Mashup (web application hybrid)
3 If lxml isn't present, riko will default to the builtin Python xml parser
4 If speedparser isn't present, riko will default to feedparser
5 Yahoo discontinued Yahoo! Pipes in 2015, but you can view what [remains](#)
6 [Huginn](#), [Flink](#), [Spark](#), and [Storm](#)
7 You can mitigate this via the [split](#) module
8 Doesn't depend on outside services like MySQL, Kafka, YARN, ZooKeeper, or Mesos
9 [Complex Event Processing](#)
10 Huginn doesn't appear to make [async web requests](#)
11 Many libraries can't parse RSS streams without the use of 3rd party libraries
12 While most libraries offer a local mode, many require integrating with a data ingestor
(e.g., Flume/Kafka) to do anything useful
13 I can't find evidence that these libraries offer an async APIs (and apparently [Spark](#)
[doesn't](#))
14 You can instead enable a ProcessPool by additionally passing threads=False to
SyncPipe, i.e., SyncPipe('fetch', conf={'url': url}, parallel=True,
threads=False).
15 the output stream of an aggregator is an iterator of only 1 item.